Section 1 **15 Min**

Q1: Write a query to display both the `ItemId` and `Category` together.

Answer :

```
SELECT ItemId, Category
FROM Menuitems;
```

Q2: **Display all menu item records from the `MenuItems` table only for items that have been ordered by customers.**

Answer:

```
SELECT DISTINCT
    m.ItemId,
    m.ItemName,
    m.Category,
    m.Price
FROM MenuItems m
INNER JOIN Orders o
  ON m.ItemId = o.ItemOrdered;
```

Q3: **Write a query to display all menu item details from the `MenuItems` table, only for items with a price higher than $10.**

Answer;

```
SELECT ItemId, ItemName,Category,Price
FROM MenuItems
WHERE Price > 10;
```

Q4: **Write a query to display all menu items which are not in the 'Pasta' category.**
Answer :

```
SELECT *
FROM MenuItems
WHERE Category != 'Pasta';
```

Q5: **Fetch the customer names and the items they ordered. Display the customer even if the ordered item is not on the menu**.
Answer:

```
SELECT o.CustomerName,
    m.ItemName
FROM Orders o
LEFT JOIN MenuItems m
ON o.ItemOrdered = m.ItemId;
```

Q6: Retrieve a list of unique ItemIds which are either in the 'Pasta' category or have been ordered by a customer.
Anwer:

```
SELECT ItemId
FROM MenuItems
WHERE Category = 'Pasta'

UNION

SELECT DISTINCT ItemOrdered AS ItemId
FROM Orders;
```

Q7: **Find the price of the second cheapest menu item**.
Anwer:

```
SELECT Price

FROM MenuItems

ORDER BY Price

LIMIT 1, 1;
```

Section 2 20 Min

1.Data type conversion, and what are the common functions used for explicit conversions? Examples

In PL/SQL, data type conversion means changing a value from one data type to another so it can be stored, compared, displayed, or used in calculations correctly.

Types of conversion

- **Implicit conversion** – Oracle automatically converts one data type to another when needed, such as comparing a number column with a numeric string.
- **Explicit conversion** – The programmer intentionally converts the value by using a conversion function.
  PL/SQL, explicit conversions are commonly performed using `TO_CHAR()`, `TO_NUMBER()`, `TO_DATE()`, and `CAST()`.

2. How do nested PL/SQL blocks work, and what are the scoping rules for variables declared in different levels of a nested blocks?

Nested PL/SQL blocks are blocks written inside another PL/SQL block. They help organize code, improve readability, and allow variables to have local scope.

Scoping rules

1. **Outer block variables**
   - Can be accessed in the outer block and all inner (nested) blocks.
2. **Inner block variables**

- Can be accessed only within that inner block.
- They are not visible outside the inner block.

3. **Variable shadowing**
   - If an inner block declares a variable with the same name as an outer block variable, the inner variable hides (shadows) the outer variable within that inner block.

3. What are the steps to use the pipeline table functions in PL/SQL to return result sets more efficiently, and what are some of the best practices for implementing them?

A **pipelined table function** is a PL/SQL function that returns rows one at a time using the `PIPE ROW` statement, instead of returning the entire collection at once. This improves performance and reduces memory usage for large result sets.

Steps to implement a pipelined table function

1. **Create an object type** to represent a row.
2. **Create a table type** as a collection of the object type.
3. **Create a function** with the `PIPELINED` keyword.
4. **Use `PIPE ROW`** to return each row.
5. **Return** from the function.
6. **Query the function** using the `TABLE()` operator.

4. How would you design a PLSQL routine to efficiently process and transform data from multiple tables into a consolidated format?

To efficiently process and transform data from multiple tables into a consolidated format in PL/SQL, I would:

1. **Use SQL JOINs** to fetch data from multiple tables.
2. **Use a cursor** to read the required records.
3. **Apply transformation logic** (calculations, formatting, filtering) inside the PL/SQL block.
4. **Use `BULK COLLECT` and `FORALL`** for large datasets to improve performance.
5. **Insert or update** the transformed data into a target table.
6. **Handle exceptions** and commit the transaction.

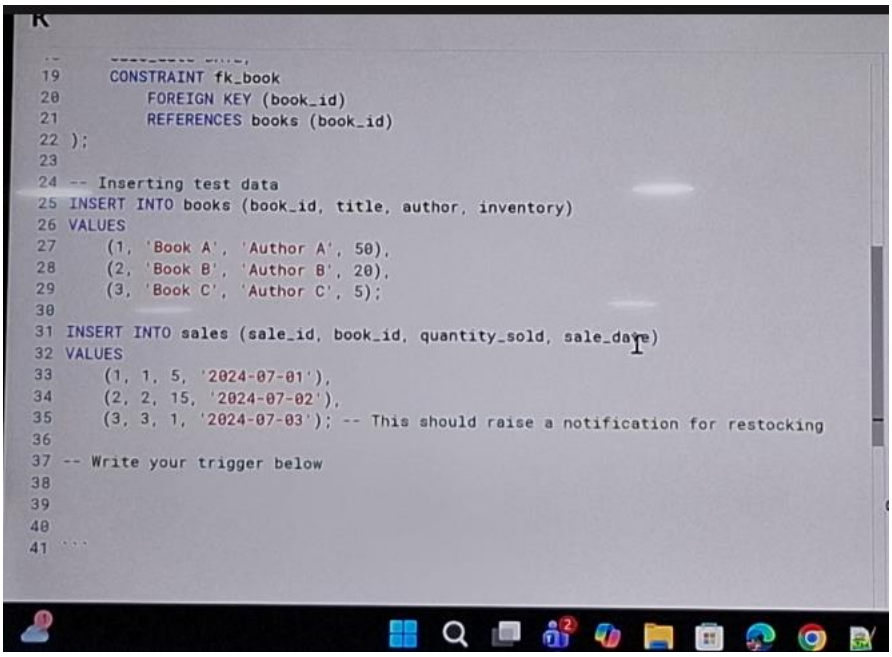
5. What are some techniques that you can use to profile and optimize a PL/SQL application that involves multiple layers of nested loops and recursive calls, ensuring efficient use of resources and minimal execution time?

To profile and optimize a PL/SQL application with nested loops and recursive calls, I would follow these techniques:

- **Use PL/SQL Profiler (`DBMS_PROFILER`)** or **Hierarchical Profiler (`DBMS_HPROF`)** to identify the sections of code that consume the most execution time.
- **Reduce nested loops** by replacing them with SQL `JOIN`s or set-based operations wherever possible.
- **Use `BULK COLLECT` and `FORALL`** to process multiple rows at once instead of row-by-row processing.
- **Create indexes** on frequently searched and joined columns to improve query performance.
- **Fetch only the required columns and rows** to reduce memory usage and execution time.
- **Optimize recursive logic** by limiting recursion depth or replacing recursion with iterative logic if appropriate.
- **Avoid repeated queries inside loops** by retrieving data once and reusing it.
- **Handle commits efficiently**, using batch commits instead of committing inside every loop iteration.
- **Review execution plans** using `EXPLAIN PLAN` to identify expensive SQL operations.

Section 3 20 Min

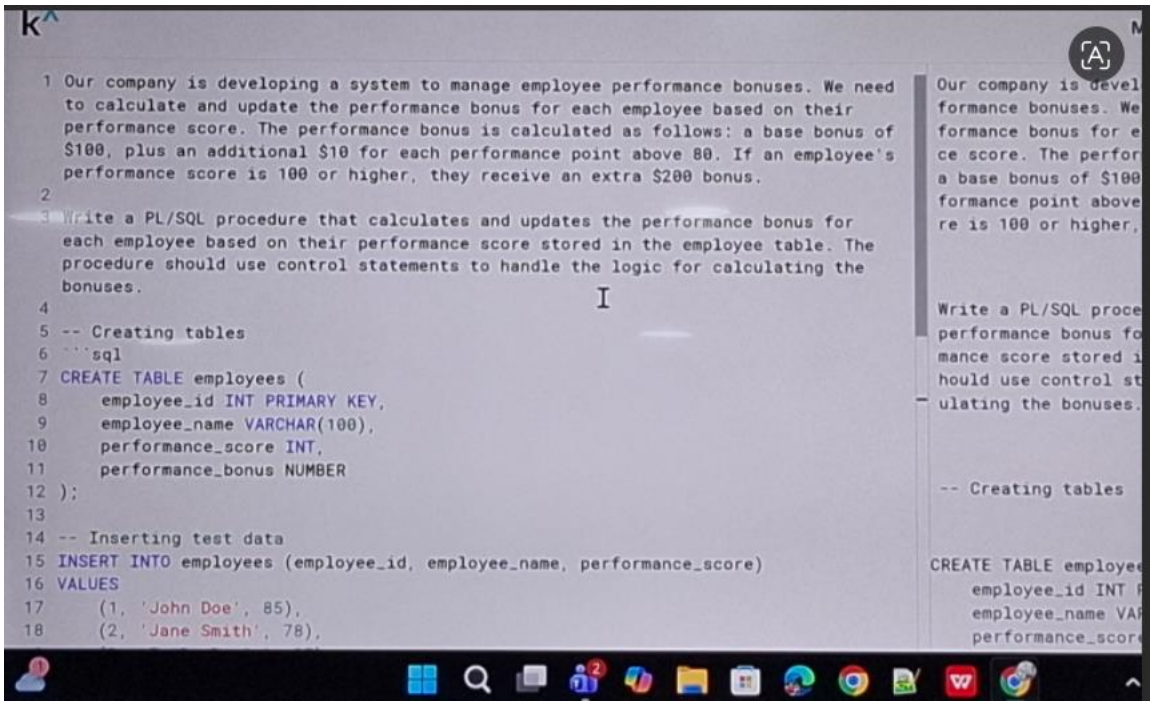
1.

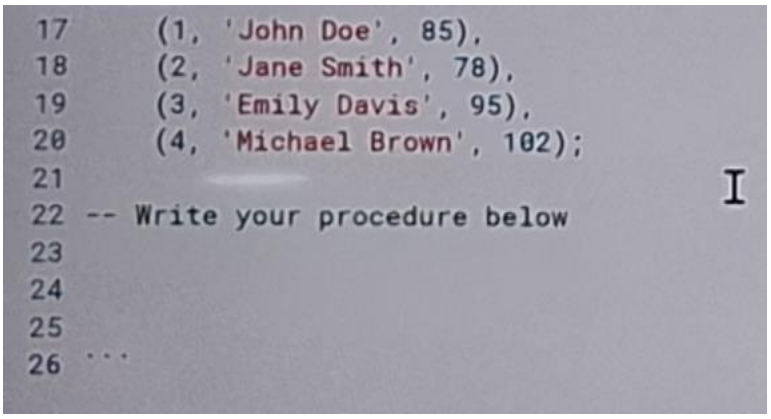


CREATE OR REPLACE TRIGGER trg_update_inventory

AFTER INSERT ON sales

```
FOR EACH ROW
DECLARE
   v_inventory books.inventory%TYPE;
BEGIN
   -- Update inventory
   UPDATE books
   SET inventory = inventory - :NEW.quantity_sold
   WHERE book_id = :NEW.book_id;
  -- Check updated inventory
   SELECT inventory
   INTO v_inventory
   FROM books
   WHERE book_id = :NEW.book_id;
   -- Raise notification if inventory is below threshold
   IF v_inventory < 10 THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Inventory below threshold. Restocking required.'
      );
   END IF;
END;
/
```





CREATE OR REPLACE PROCEDURE calculate_performance_bonus

IS

v_bonus NUMBER;

BEGIN

FOR rec IN (

SELECT employee_id, performance_score

FROM employees

```
    )
    LOOP
        -- Base bonus
        v_bonus := 100;
        -- Bonus for points above 80
        IF rec.performance_score > 80 THEN
            v_bonus := v_bonus + (rec.performance_score - 80) * 10;
        END IF;
        -- Extra bonus for score >= 100
        IF rec.performance_score >= 100 THEN
            v_bonus := v_bonus + 200;
        END IF;
        UPDATE employees
        SET performance_bonus = v_bonus
        WHERE employee_id = rec.employee_id;
    END LOOP;

    COMMIT;
END;
/
```